# ROS 2 Differential Drive Bot Hackathon: Line Follower Challenge

## 1. The Challenge (Problem Statement)

This hackathon challenges participants to design, simulate, and program a simulated **differential drive robot** to accurately follow a predefined line. The event will be conducted in a **ROS 2 Gazebo simulation environment**, focusing on core robotics concepts including URDF modeling, Gazebo simulation plugins (or `ros2_control`), and feedback-based control systems. Teams will be graded on the accuracy of their robot's path following.

The challenge is divided into **two phases** to test development and adaptability:

## Phase 1: The Known Path (Open Phase)

- A pre-defined line path (and some basic shapes) will be provided at the start of the hackathon.
- Teams will have ample time to develop and tune their line-following algorithms to achieve maximum accuracy on this path.

## Phase 2: The Unknown Path (Grading Phase)

- Three days before the hackathon ends, a new, undisclosed line path will be released.
- Teams must adapt their existing line-following node to successfully navigate this new path with high accuracy.
- This phase will be used for the final grading.

## 2. Tools and Technologies

Participants are expected to use:

- **Modeling:** Onshape for creating the robot model (Participants can use other software if they want to).
- **Simulation:** Gazebo and ROS 2. The official grading `.sdf` world file will include only the standard system plugins (`gz-sim-physics-system`, `gz-sim-sensors-system`, `gz-sim-scene-broadcaster-system`, and `gz-sim-user-commands-system`).
  Teams must ensure their robot's URDF, plugins, and ROS bridges function within this baseline environment without requiring custom world-level modifications.
- **Control:** Gazebo simulation plugins (or `ros2_control` if preferred) for interfacing with the simulated robot motors.
- **Software:** A custom ROS 2 package for the line-following logic.

## 3. The Build Process

Teams will create their own differential drive robot from scratch within the simulation environment.

## CAD & Robot Modeling

Teams are required to represent a **basic differential drive robot** either through a **CAD model** or directly by writing a **URDF description.**

### Minimum Requirements

- A chassis
- Two-wheel drive or four-wheel drive configuration (teams may choose either, but must justify the choice in the final presentation).
- Designated space for sensors (e.g., a camera or a line detection sensor placeholder)
- Exported Unified Robot Description Format (URDF) file, which must be ROS 2-compatible
- Size of chassis (without including the wheels) must not

exceed 150mmx150mm in length and breadth and height must not exceed 40mm.
- Design must be optimized for purpose, for example a box with wheels might be fine for simulation but not acceptable for final submission for CAD model.

### Guidelines

- For teams using **Onshape,** the *Onshape-to-Robot* extension may be used to convert CAD models into URDF.
- Alternatively, teams may author their URDF directly without utilizing CAD software for the simulation; executing this initial step prior to incorporating a CAD model into Gazebo is recommended.

### Bonus Provision

Teams that create a **CAD model of the robot** will receive bonus credit. Along with the URDF, the **layout of the CAD model** should also be submitted. CAD-based designs will be evaluated on **design quality, compactness, and overall feasibility**.

Bonus credit will be given if you submit engineering drawings for the final submission.

## ROS 2 Integration

- Load your robot into the **Gazebo simulation environment** with the generated URDF.
- Integrate Gazebo simulation plugins or `ros2_control` to interface the wheel joints with velocity commands.
- Configure the necessary `ros_gz_bridge` parameters (if using plugins) or the controller manager YAML files (if using `ros2_control`) to enable motor control and state publishing.

## Applied Electronics (AE) – Parts List & Circuit Diagram

- Teams must prepare a **parts list** of components required to build a physical version of the line-follower robot.
- The list should include (but is not limited to): motors, wheels, chassis materials, sensors (e.g., IR sensors or camera), microcontroller (e.g., Arduino or Raspberry Pi), motor drivers, power supply, and any wiring or structural elements.
- Teams will be evaluated on the **completeness**, **accuracy**, and **justification** of this parts list as part of the AE domain.
- **Bonus credit** will be awarded to teams that provide a **well-labeled circuit diagram** illustrating the connections between the major components.

# 4. Grading Criteria and Technical Constraints

- Grading will be based on the **accuracy of the robot's path following**.
- A provided **grading node** will measure the error between the robot's actual position and the ideal line path.
- Grading rubric **will not be changed** under any circumstances.

## The Grading Node

- The node will subscribe to a topic that broadcasts the position and orientation of your robot.
- It will have access to the ideal waypoints that make up the line path.
- Using this data, the node will continuously calculate the **real-time error**, measuring the distance and orientation difference between your bot's current position and the nearest point on the ideal path.
- This data will be logged and used to calculate a final accuracy score for your team.

## 5. Submission Requirements

Teams must submit a single **ROS 2 workspace** containing:

- **Robot URDF:** The model of their differential drive bot, including its links, joints, and sensors.
- **Control Configuration:** The URDF/plugin configuration or `ros2_control` YAML file used for the robot's motor interface.
- **ROS 2 Package:** A complete package containing their line-following node.
- **AE Part list:** A complete list containing the parts required in the bot.
- **Submissions for bonus credit**(*optional)*

***FINAL Presentation:*** *31st October*